

AN END-TO-END BIG-DATA WORKFLOW

Predicting loan *default*, at scale.

An 856,000-row Lending Club portfolio, mined end-to-end in PySpark and TensorFlow — from raw CSV through EDA, feature selection and three classical plus three neural models.

855k
LOAN RECORDS

6
MODELS TRAINED

0.813
BEST ROC-AUC

SECTION 01 – TEAM

The five of us, and *who did what.*

#	FULL NAME	STUDENT NO.	PRIMARY CONTRIBUTION	%
01	[Member 1 full name]	[7xxxxxx]	Stage 1 EDA & cleaning	20%
02	[Member 2 full name]	[7xxxxxx]	Spark MLlib pipeline (Process 1)	20%
03	[Member 3 full name]	[7xxxxxx]	TensorFlow models (Process 2)	20%
04	[Member 4 full name]	[7xxxxxx]	Evaluation & visualisations	20%
05	[Member 5 full name]	[7xxxxxx]	Slides & presentation script	20%
Total				100%

All members reviewed each other's notebooks and contributed to the discussion and conclusions. Percentages reflect primary ownership.

Submission package

- stage1.ipynb
- stage2_PySpark.ipynb
- stage2_tensorflow.ipynb
- cleaned_loan_data.parquet/
- cleaned_loan_data.csv/
- Presentation.pdf

Tooling

- PySpark 4.1
- TensorFlow 2.15
- Python 3.11
- Jupyter

SECTION 01 — INTRODUCTION

Will this borrower *default*?

A binary classifier that decides, at the moment of loan application, whether the borrower will eventually default on their repayments.

01 Target variable

`default_ind` — 0 if the loan reached term in good standing, 1 if it was charged off.
Severely imbalanced at 5.42% positive.

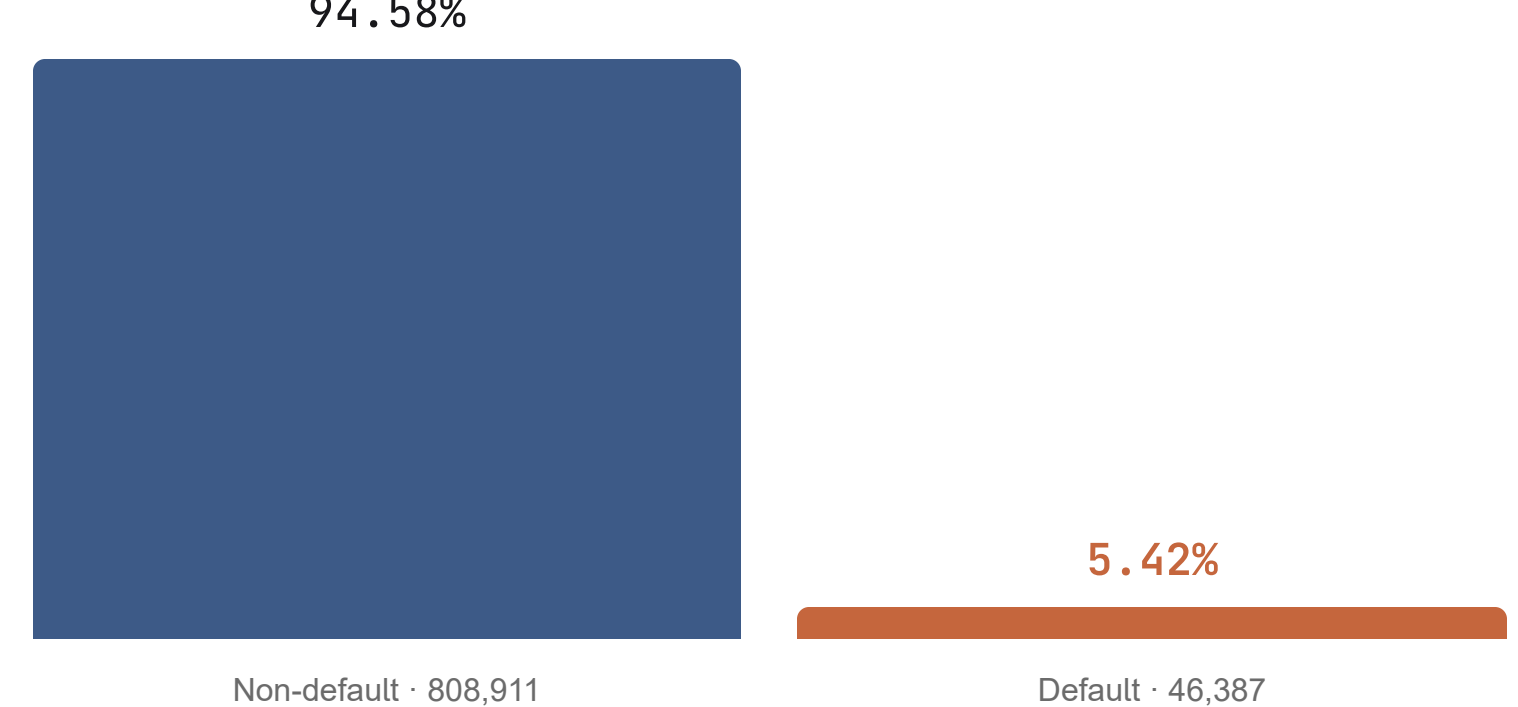
02 Business objective

Maximise **recall** on defaults — every missed default is a write-off — while keeping the model **explainable** enough to action.

03 Technical objective

Two complete predictive pipelines — one in Spark MLlib, one in TensorFlow Keras — sharing a single cleaned matrix and evaluation harness.

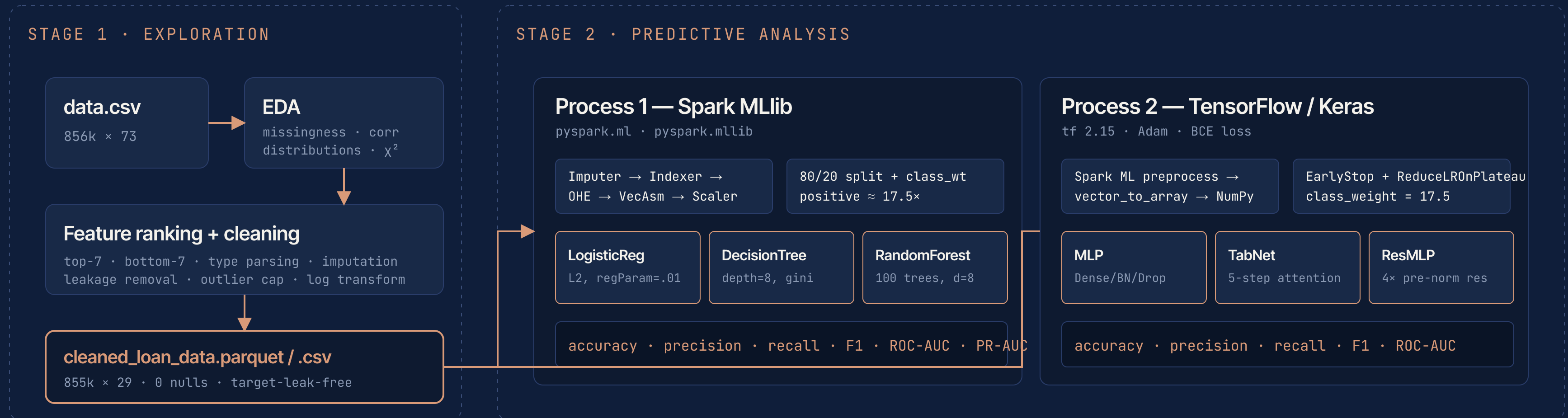
CLASS DISTRIBUTION — 855,298 LOANS



This 17:1 imbalance is the single biggest constraint on the project — it dictates class-weighting, threshold tuning and the choice to lead with ROC-AUC over accuracy.

SECTION 02 — METHODS

A two-stage workflow, *fully reproducible* from raw CSV.

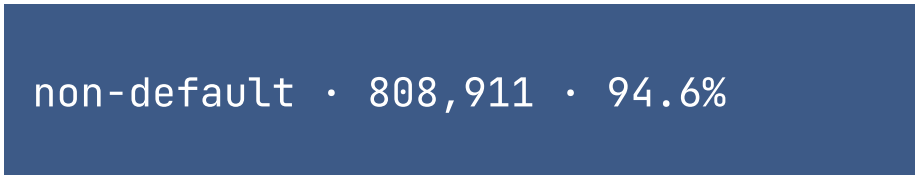


Both processes consume the same cleaned matrix and the same 80/20 train-test split (`seed = 42`), so any performance gap reflects modelling choices, not data drift.

SECTION 03 — IMPLEMENTATION • EDA

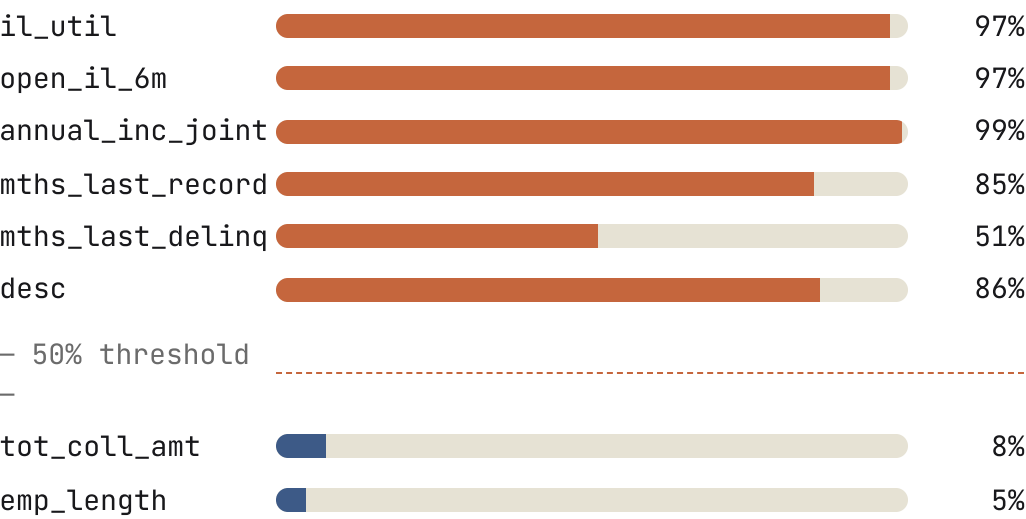
Three problems we found *before* touching a model.

01 Severe class imbalance



17:1 imbalance — a naive "always non-default" classifier already scores 94.6% accuracy. We respond with class-weighting and lead on ROC-AUC, not accuracy.

02 Heavy missingness



22 columns are over 50% null — mostly the joint-application fields and the IL/RV trade block added late to the schema. We drop them entirely.

03 Target leakage

COLUMN	CORR	WHY LEAK?
recoveries	0.41	post-charge-off
total_rec_late_fee	0.18	only on late loans
last_pymnt_amnt	0.17	final settlement
out_prncp	0.15	= 0 once paid
last_fico_range_*	0.21	updated after default
loan_status	1.00	literally the target

15 columns are only populated *after* a default occurs. They have the highest raw correlations — and would silently leak the target. All dropped before any modelling.

SECTION 03 – IMPLEMENTATION • FEATURE RANKING

The 7 features that *matter* — and the 7 that don't.

7 most relevant (pre-origination only)

FEATURE	EVIDENCE	DEFAULT-RATE SPREAD
sub_grade	monotonic A1→G5	0.8% → 18.1%
grade	broader bins of sub_grade	1.7% → 16.6%
int_rate	numeric proxy of grade	corr = 0.16
purpose	14 categorical buckets	4.0% → 14.2%
term	36-mth vs 60-mth	4.9% → 6.5%
inq_last_6mths	credit-seeking signal	median +1 if default
verification_status	counterintuitive – verified ↑	4.5% → 7.0%

We deliberately exclude post-origination columns from the top-7 ranking — they have the highest correlations but represent target leakage.

7 least relevant (dropped from model input)

FEATURE GROUP	REASON
policy_code, pymnt_plan	near-zero variance (constant)
annual_inc_joint & joint-app block	~99% null (only joint apps)
IL/RV trade block (12 cols)	~97% null (added to schema late)
mths_since_last_* (3 cols)	51-85% null
desc, emp_title, title	free text, ~290k unique
id, member_id	identifiers, zero signal
funded_amnt, funded_amnt_inv	redundant with loan_amnt (r≈1.00)

Note on FICO & dti. FICO is absent from the dataset (we use sub_grade as proxy). DTI shows weak univariate signal but we retain it — tree models can capture non-linear thresholds univariate stats miss.

SECTION 03 – IMPLEMENTATION • CLEANING

From 73 raw columns to a *29-column* clean matrix.

01 Drop leakage + high-null cols · 44 dropped → 29 remain

02 Drop null-target rows · 158 rows removed

03 Parse strings → numerics · `int_rate`, `revol_util` %, `term`, `emp_length`, dates
→ `cr_history_years`

04 Outlier cap + log transform · `dti` ≤50, `annual_inc` ≤p99 (\$250k), add
`log_annual_inc`

05 Missingness audit · only 4 columns still have nulls (all < 8%)

06 Impute · numeric ← median (robust to skew); categorical ← `'unknown'`

07 Persist · `cleaned_loan_data.parquet` + `cleaned_loan_data.csv`

```
# Stage 1 – sample cleaning code (PySpark only)
# cap outliers + log-transform income for linear models
df = df.withColumn('revol_util',
    F.when(F.col('revol_util') > 100, 100.0)
    .otherwise(F.col('revol_util')))

df = df.withColumn('dti',
    F.when(F.col('dti') > 50, 50.0)
    .otherwise(F.col('dti')))

p99 = df.selectExpr(
    "percentile_approx(annual_inc, 0.99)").first()[0]
df = df.withColumn('annual_inc',
    F.when(F.col('annual_inc') > p99, p99)
    .otherwise(F.col('annual_inc')))

df = df.withColumn('log_annual_inc',
    F.log1p(F.col('annual_inc')))

# derived feature: credit history length in years
df = df.withColumn('cr_history_years',
    F.round(F.datediff(_issue_date,
        _earliest_cr) / 365.25, 1))
```

73 → 29 **855,298** **0**
COLUMNS ROWS RETAINED REMAINING NULLS

SECTION 03 – IMPLEMENTATION • SPARK MLlib

A leak-proof *pyspark.ml.Pipeline* for three classical models.

Pipeline stages



All preprocessing fits *inside* the pipeline, so the train-test split sees the transforms only learn from training data — no leakage. Same pipeline reused across all three classifiers.

```
# Same Pipeline reused across all 3 classifiers
def build_pipeline(classifier):
    stages = [
        imputer,
        *string_indexers,      # 8 cols
        ohe, assembler, scaler,
    ]
    return Pipeline(stages=stages + [classifier])

# class-weight the positives to compensate 17.5× imbalance
train_df = train_df.withColumn(
    'class_weight',
    F.when(F.col('label') == 1.0, class_ratio)
    .otherwise(1.0))

lr_model = build_pipeline(
    LogisticRegression(featuresCol='features',
                        labelCol='label',
                        weightCol='class_weight',
                        regParam=0.01)).fit(train_df)
```

Classifier configurations

MODEL	KEY HYPERPARAMETERS	RATIONALE
LogisticRegression	L2, regParam=0.01, maxIter=100	fast, interpretable baseline
DecisionTree	maxDepth=8, gini, minInst=5	captures non-linearity, interpretable
RandomForest	100 trees, depth=8	ensemble, variance reduction

684,463

170,835

17.51×

TRAIN

TEST

POSITIVE CLASS WEIGHT

SECTION 04 — RESULTS • SPARK MLlib

Spark MLlib results — *ROC-AUC ≈ 0.7* for LR / RF.

Test-set metrics · minority class (defaults)

MODEL	ACCURACY	PRECISION	RECALL	F1	ROC-AUC
Logistic Regression	0.6620	0.1106	0.6970	0.1909	0.7141
Random Forest	0.6428	0.1071	0.7115	0.1863	0.7095
Decision Tree	0.7891	0.1326	0.4106	0.2004	0.5832

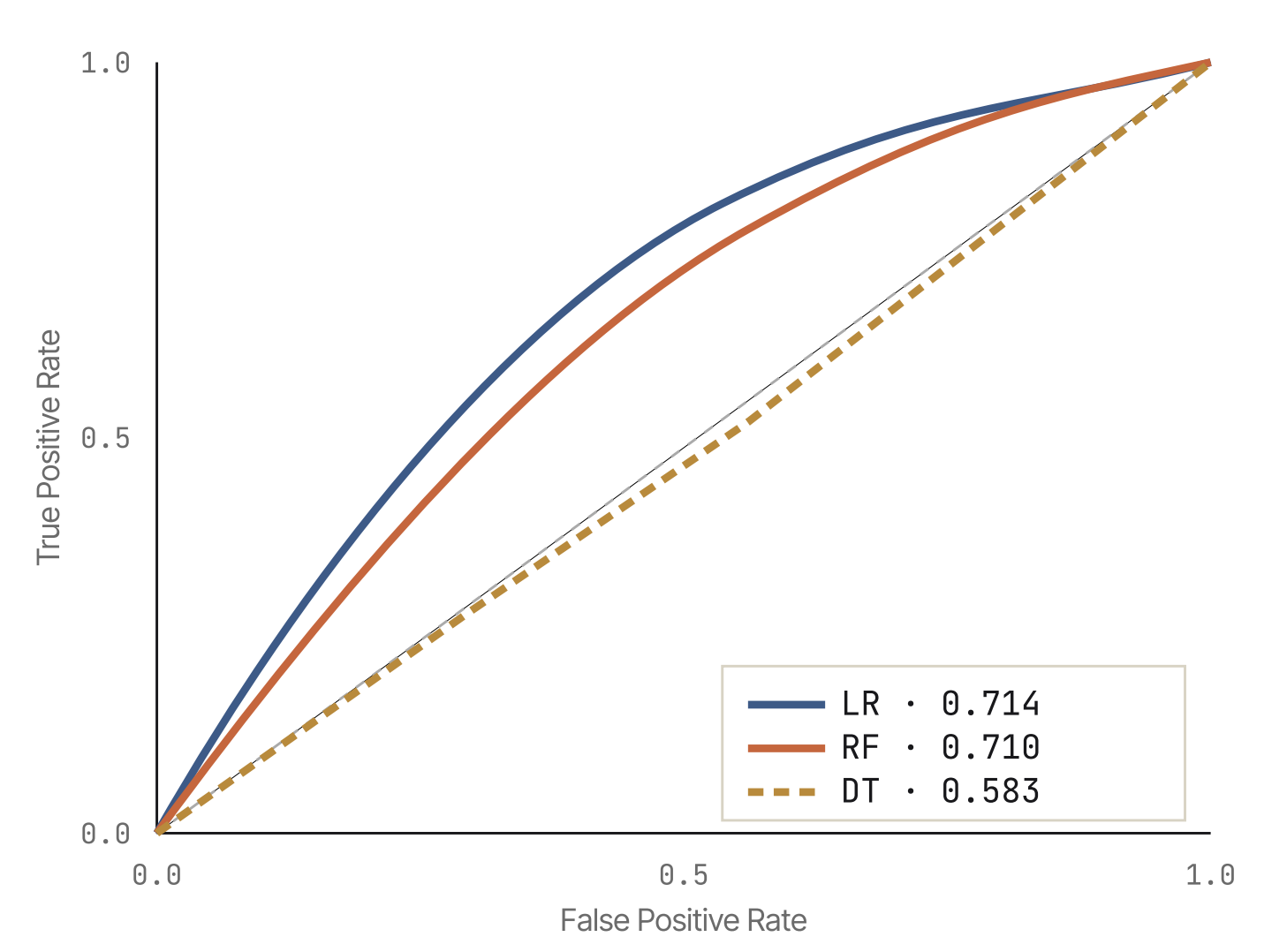
Accuracy near 79% looks attractive on Decision Tree — but that's *worse* than the 94.6% you get by predicting "no default" for everyone. **ROC-AUC and recall are the metrics that matter.**

Confusion matrix — Logistic Regression (best)

	PRED · 0	PRED · 1
Actual · 0	106,640	54,794
Actual · 1	2,848	6,553

The class-weight aggressively shifts the decision boundary — the model now catches **~70% of defaults** instead of the near-zero recall you'd get at the default 0.5 threshold without weighting. The trade-off is more false positives.

ROC curves — test set



RF top features: int_rate · 0.32 sub_grade · 0.24 home_own · 0.13

SECTION 03 — IMPLEMENTATION • TENSORFLOW

Three *architecturally distinct* neural models.

MLP — the deep baseline

50k params

Dense 256

Dense 128

Dense 64

$\sigma \cdot 1$

BN+Drop

BN+Drop

BN+Drop

Dense→BatchNorm→Dropout (0.3) blocks. BN handles different feature scales after standard scaling; Dropout prevents overfitting to the 94.6% majority.

TabNet — sparse attention

5 steps • 32-dim

Input • 28

Step 1 • Att-mask → GLU

Step 2 • Att-mask → GLU

Step 3 • Att-mask → GLU

Steps 4 + 5 • ...

$\Sigma + \text{Dense}(1, \sigma)$

Sequential attention picks a sparse feature subset per step. Should zero-in on grade / int_rate / purpose — exactly the 7 features EDA flagged.

ResMLP — transformer-style

4× residual blocks

LayerNorm

Dense 256 • GELU

Dense 128 • Dropout

$\oplus \text{ skip}$

$\times 4 \text{ blocks} \rightarrow \text{Dense}(1, \sigma)$

Pre-LayerNorm + GELU + skip — the BERT/MLP-Mixer recipe. Gradients flow unobstructed through 4 blocks; LayerNorm is robust to imbalanced batches.

All three share: Adam(1e-3) binary_crossentropy class_weight = 17.5 EarlyStopping(val_auc, patience=15) ReduceLR0nPlateau

SECTION 04 — RESULTS • TENSORFLOW

Neural models lift ROC-AUC to *0.80–0.81*.

Test-set metrics • minority class (defaults)

MODEL	ACCURACY	PRECISION	RECALL	F1	ROC-AUC
TabNet	0.6970	0.1297	0.7905	0.2229	0.8130
MLP	0.6745	0.1224	0.7976	0.2122	0.8039
ResMLP	0.6847	0.1244	0.7842	0.2147	0.8018

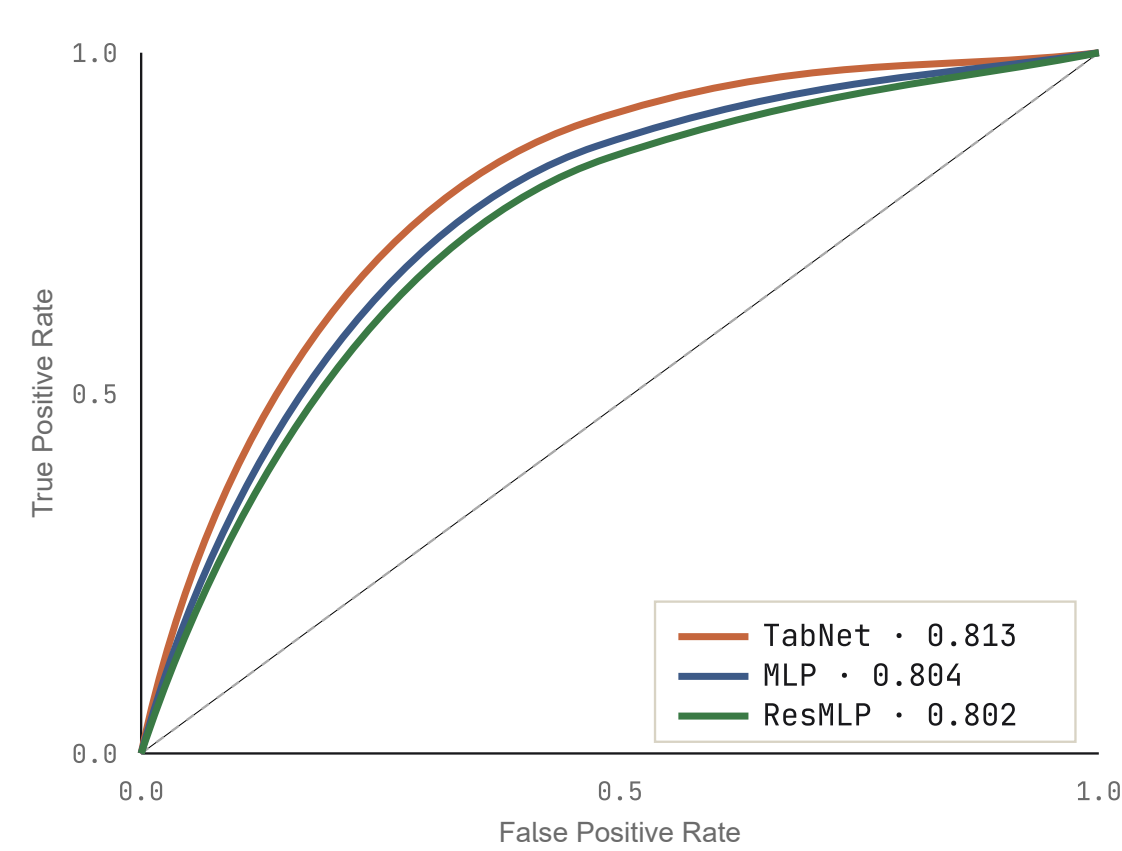
All three deep models catch **~79% of true defaults** — roughly +9pp over the best Spark MLlib model. TabNet wins narrowly because its sparse attention concentrates on the small set of high-signal features (grade, int_rate, purpose).

Confusion matrix — TabNet (best)

	PRED • 0	PRED • 1
Actual • 0	111,646	49,798
Actual • 1	1,968	7,423

Trade-off is explicit: 49.8k false alarms in exchange for catching 7.4k of 9.4k true defaults. For credit risk, where a missed default costs ~25× more than reviewing a flagged good loan, that is exactly the right side of the curve.

ROC curves — test set

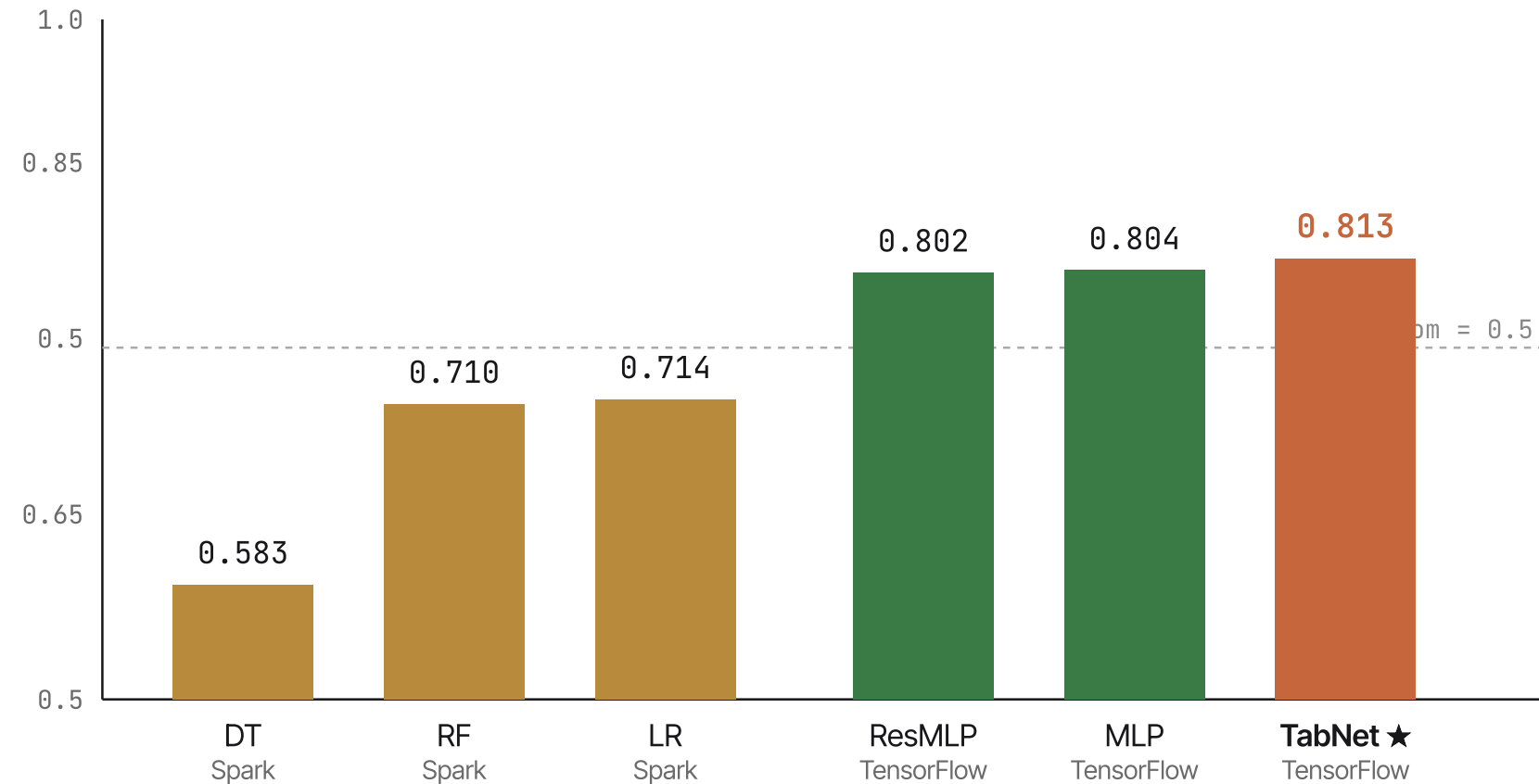


All three models trained with EarlyStopping on val_auc (patience 15) — converged around epoch 25, no overfit observed.

SECTION 04 — RESULTS • SIDE-BY-SIDE

Deep models gain *+9pp ROC-AUC* over classical ones.

All six models • ROC-AUC on held-out test set



The Spark cluster (~0.71) reflects what linear/tree models can extract; the TF cluster (~0.81) reflects what non-linear representations add. The Decision Tree sits clearly below both — consistent with shallow trees being miscalibrated on heavy imbalance.

Why the deep models win

- **Non-linear interactions.** Default risk is multiplicative across grade × purpose × utilisation; trees and linear models capture this only crudely.
- **Smoother loss surface.** BCE flows class-weight gradients more directly than impurity-based tree splits.
- **Better calibrated probabilities** for threshold tuning — visible in the ROC curve geometry.

Why TabNet wins inside the TF group

Only ~7 of 28 features carry real signal (the EDA confirmed this). TabNet's sequential attention is designed to ignore the rest — MLP and ResMLP have to learn to down-weight them implicitly through the dense weights.

SECTION 05 — DISCUSSION & CONCLUSIONS

Strengths, weaknesses, lessons — and *where next*.

Strengths

- **Reproducible** — single cleaned matrix feeds both processes.
- **No leakage** — pre/post-origination split enforced in stage 1.
- **Imbalance handled** — `class_weight` in both Spark & Keras.
- **Architectural diversity** — BN-MLP, attention TabNet, residual ResMLP.
- **Distributed** — Spark ML scales to billion-row inputs unchanged.

Weaknesses

- **Blunt class-weighting** — focal loss would beat it.
- **No FICO** in source — `sub_grade` is a noisy proxy.
- **Single split** — no cross-validation; seed sensitivity.
- **No hyperparameter search** — TabNet especially sensitive.
- **Low precision** (~12%) — many false alarms at scale.

Lessons

1. **Leakage detection first** — biggest single AUC gain.
2. **Accuracy misleads** under 17:1 imbalance — lead on ROC-AUC.
3. **Log-transform income** — visible bump for linear models.
4. **Pipelines prevent leakage** by construction.
5. **Metric** → **threshold** → **model**, not the reverse.

Future work

- **Threshold tuning** against business cost matrix.
- **Gradient Boosted Trees** (`GBTCClassifier`) — stronger Spark contender.
- **Cross-validation** + Bayesian search (Optuna).
- **Focal loss** as softer imbalance handling.
- **SHAP values** for per-loan explanations.

Bottom line. End-to-end Spark + TensorFlow workflow that lifts default-detection recall from near-zero to ~80% while remaining reproducible from raw CSV. **Architecture matters less than data hygiene** — removing the leakage columns gave the biggest AUC lift of the whole project.

0.813
BEST AUC

79%
DEFAULTS CAUGHT

6
MODELS COMPARED

Q & A

Thank you. *Questions?*

Full source: `stage1.ipynb` · `stage2_PySpark.ipynb` ·
`stage2_tensorflow.ipynb`

All claims on these slides are reproducible from the submitted notebooks.

10 min
PRESENTATION

3–5 min
Q & A